

A Unified Gradient Regularization Family for Adversarial Examples

Chunchuan Lyu

Department of Computer Science
and Software Engineering
Xi'an Jiaotong-Liverpool University
Email: chunchuan.lv@gmail.com

Kaizhu Huang

Department of Electrical
and Electronic Engineering
Xi'an Jiaotong-Liverpool University
Email:Kaizhu.Huang@xjtlu.edu.cn

Hai-Ning Liang

Department of Computer Science
and Software Engineering
Xi'an Jiaotong-Liverpool University
Email:haining.liang@xjtlu.edu.cn

Abstract—Adversarial examples are augmented data points generated by imperceptible perturbation of input samples. They have recently drawn much attention with the machine learning and data mining community. Being difficult to distinguish from real examples, such adversarial examples could change the prediction of many of the best learning models including the state-of-the-art deep learning models. Recent attempts have been made to build robust models that take into account adversarial examples. However, these methods can either lead to performance drops or lack mathematical motivations. In this paper, we propose a unified framework to build robust machine learning models against adversarial examples. More specifically, using the unified framework, we develop a family of gradient regularization methods that effectively penalize the gradient of loss function w.r.t. inputs. Our proposed framework is appealing in that it offers a unified view to deal with adversarial examples. It incorporates another recently-proposed perturbation based approach as a special case. In addition, we present some visual effects that reveals semantic meaning in those perturbations, and thus support our regularization method and provide another explanation for generalizability of adversarial examples. By applying this technique to Maxout networks, we conduct a series of experiments and achieve encouraging results on two benchmark datasets. In particular, we attain the best accuracy on MNIST data (without data augmentation) and competitive performance on CIFAR-10 data.

Keywords—Adversarial examples; Deep learning; Regularization; Robust classification

I. INTRODUCTION

Imperceptible perturbations in images are able to change the prediction of neural network models, including single layer softmax model [1], [2], [3]. That is, given a trained neural network model and an input image, one can always find a small perturbation that can change the model's predication through certain optimization procedure. Moreover, perturbations trained in one model could also change the prediction results of many other classification models. The examples constructed by using such perturbations are referred to as *adversarial examples*, which have attracted much interest in both machine learning and data mining [2], [4], [5].

The existence of adversarial examples raises several important issues. First, why do such perturbations exist in the first place? Intuitively, a successful machine learning model should robustly classify indistinguishable inputs as the same class. The

generalizability of those perturbations is even more intriguing, since those perturbations are obtained by optimization process based on model and image instance. Second, why do such perturbations not occur frequently in real applications (i.e., naturally generated data)? Indeed, if most examples are adversarial examples, no machine learning algorithm could work at all. Finally and most importantly, what can we do to deal with adversarial examples? One possibility is to build machine learning models that are immune to adversarial examples. Alternatively, we might be able to use adversarial examples to even improve the performance of most machine learning models.

To explain the existence of adversarial examples, [2] argued that this particular phenomenon could arise naturally from high dimensional linearity, as opposite to nonlinearity suspected by [1]. [6] showed that models' robustness against adversarial examples are limited by distinguishability between classes. This also supports that nonlinearity is not the fundamental reason behind adversarial examples. They also argued the generalizability of adversarial examples is due to neural network models' resemblance of linear classifiers. In addition to directly addressing the problem, [3] have discovered a twin problem: there exist human unrecognizable images, which deep learning models could classify them with high confidence. There is not yet a verifiable answer to why adversarial examples exist only infrequently in real-life cases, but some speculate that they appear only in low probability regions in data manifolds [1], [2]. To alleviate the influence caused by adversarial examples, [7] tried to penalize the Jacobian matrix based on a series of approximations. While their model seems more robust, it usually leads to an accuracy drop and slow training due to additional cost. Accuracy can be improved by injecting those perturbed examples back in training, as demonstrated by [1]. [2] have further extended this idea to the so-called *fast gradient sign method*, and when using this method, trained models are more robust against adversarial examples. While this method seems promising, it appears to lack mathematical motivation and failing to fully utilize the idea of linear perturbation as a consequence. In data mining community, researchers also try to build models that are robust against adversarial examples (adversarial attacks) [4], [5]. In particular, a study investigated correspondence between adversarial examples and effective regularizer, but their results are restricted merely for some specific loss functions [4].

We develop the linear view of adversarial examples pro-

posed by [2] in a more rigorous and unified way. To this end, we propose a unified framework to train models that is robust to adversarial examples and successfully transform it to a minmax problem. Specifically, we propose a family of gradient based perturbations, as a unified regularization technique. Models trained by applying our proposed gradient regularization family have proved highly robust to perturbations. Moreover, under different values of norm parameter, the family presents itself a unified framework, i.e.- it incorporates the recently-proposed *fast gradient sign method* [2] as a special case and can also derive many other more promising methods. One interesting special case is verified to achieve encouraging performance in two benchmark data sets. Furthermore, by magnifying perturbations in MNIST [8], we could provide physical intuition for why adversarial examples could be generalized across different machine learning models.

II. GRADIENT REGULARIZATION FAMILY

In this section, we will present our framework of *gradient regularization family* and describe its theoretical properties.

A. General Framework

We introduce our framework by starting with the worst-case perturbation, initially proposed in [2]. One salient feature of our framework is that it can represent a significant extension to the previous work into a unified family that incorporates many important variants of gradient regularization technique.

Denote $\mathcal{L}(\mathbf{x}; \boldsymbol{\theta})$ ¹ as a loss function, $\mathbf{x}$ as data, and $\boldsymbol{\theta}$ as model parameters. The idea could be formalized as follows. Instead of solving $\min_{\boldsymbol{\theta}} \mathcal{L}(\mathbf{x}; \boldsymbol{\theta})$, ideally we would like to solve the following problem if we try to build a robust model against any small perturbation defined as $\boldsymbol{\epsilon}$:

$$\min_{\boldsymbol{\theta}} \max_{\boldsymbol{\epsilon}: \|\boldsymbol{\epsilon}\|_p \leq \sigma} \mathcal{L}(\mathbf{x} + \boldsymbol{\epsilon}; \boldsymbol{\theta}) \quad (1)$$

The norm constraint in the inner problem implies that we only require our model to be robust against certain small perturbation. Thus the training procedure is decomposed into two stages. We first find a perturbation that maximizes the loss given the data and constraints. Then, we perform our ordinary training procedure to minimize the loss function by altering $\boldsymbol{\theta}$. In general, this problem is difficult to be solved due to its non-convex nature with respect to $\boldsymbol{\epsilon}$ and $\boldsymbol{\theta}$.

In the following, we will propose to solve the problem using approximation technique. To this end, we first approximate the loss function by its first order Taylor expansion at point $\mathbf{x}$. The inner problem then becomes:

$$\max_{\boldsymbol{\epsilon}} \mathcal{L}(\mathbf{x}) + \nabla_{\mathbf{x}} \mathcal{L}^T \boldsymbol{\epsilon} \quad \text{s.t.} \quad \|\boldsymbol{\epsilon}\|_p \leq \sigma \quad (2)$$

This problem is trivially linear, and hence convex w.r.t. $\boldsymbol{\epsilon}$. We can obtain a closed form solution by Lagrangian multiplier method, see A for details. This yields:

$$\boldsymbol{\epsilon} = \sigma \text{sign}(\nabla \mathcal{L}) \left(\frac{\|\nabla \mathcal{L}\|}{\|\nabla \mathcal{L}\|_{p^*}} \right)^{\frac{1}{p-1}} \quad (3)$$

where p^* is the dual of p , i.e., $\frac{1}{p^*} + \frac{1}{p} = 1$.

If we substitute the optimal $\boldsymbol{\epsilon}$ back to the original optimization problem, we can see that the influence of perturbations can be formulated as a regularization term. Thus, the new family of regularization method works approximately as:

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\mathbf{x}) + \sigma \|\nabla_{\mathbf{x}} \mathcal{L}\|_{p^*} \quad (4)$$

Instead of minimizing $\mathcal{L}(\mathbf{x})$, we try to minimize $\mathcal{L}(\mathbf{x} + \boldsymbol{\epsilon})$ where $\boldsymbol{\epsilon}$ is defined above, and parameterized by p such that the new optimization objective could be highly robust to any small perturbations.

Remark 1. It is worthwhile to note that although it has been long known in [9], that injecting Gaussian noise is equivalent to penalizing the trace of Hessian matrix, to our best knowledge, our approach is the first general method to penalize the gradient of loss function w.r.t. the input in complex models. Such regularization could be applied into various machine learning model that contains gradient information.

In the following, we examine three special cases of this family and show how this unified regularization framework could contain another method [2].

B. Case $p = \infty$

We show that in this case our method can be reduced to the *fast gradient sign method* proposed in [2]. The worst perturbation $\boldsymbol{\epsilon}$ becomes:

$$\boldsymbol{\epsilon} = \sigma \lim_{p \rightarrow \infty} \text{sign}(\nabla \mathcal{L}) \left(\frac{\|\nabla \mathcal{L}\|}{\|\nabla \mathcal{L}\|_{p^*}} \right)^{\frac{1}{p-1}} \quad (5)$$

$$= \sigma \text{sign}(\nabla \mathcal{L}) \left(\frac{\|\nabla \mathcal{L}\|}{\|\nabla \mathcal{L}\|_1} \right)^0 \quad (6)$$

$$(\text{assuming } \|\nabla \mathcal{L}\| > 0) \quad (7)$$

$$= \sigma \text{sign}(\nabla \mathcal{L}) \quad (8)$$

The corner case, where $\|\nabla \mathcal{L}\| = 0$, defines $\text{sign}(0)$ as 0. Therefore, we can reduce our general method to a special case, namely *fast gradient sign method*. The corresponding induced regularization term is $\sigma \|\nabla \mathcal{L}\|_1$. Mathematically speaking, this regularization term appears to be unnatural, since the gradient is not penalized in an isotropic way. This might affect negatively the performance, especially when the data is preprocessed to be Gaussian-like.

C. Case $p = 1$

This is another special case, where $p = 1$, and thus the worst perturbation $\boldsymbol{\epsilon}$ becomes:

$$\boldsymbol{\epsilon} = \sigma \lim_{p \rightarrow 1} \text{sign}(\nabla \mathcal{L}) \left(\frac{\|\nabla \mathcal{L}\|}{\|\nabla \mathcal{L}\|_{p^*}} \right)^{\frac{1}{p-1}} \quad (9)$$

$$= \sigma \text{sign}(\nabla \mathcal{L}) \left(\frac{\|\nabla \mathcal{L}\|}{\|\nabla \mathcal{L}\|_{\infty}} \right)^{\infty} \quad (10)$$

Therefore,

$$\boldsymbol{\epsilon}_i = \begin{cases} \sigma, & \nabla \mathcal{L}_i = \max_j \mathcal{L}_j \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

The intuition here is quite clear. Since we are constrained by the sum of absolute value of all perturbations, it is intuitive to put all of our “budgets” into one direction. However, the induced regularization term $\|\nabla \mathcal{L}\|_{\infty}$ is not very appealing, since it only penalizes gradient in one direction.

¹This notation might be shorthand as $\mathcal{L}(\mathbf{x})$ or $\mathcal{L}$ in the following text.

D. Case $p = 2$

As [10] have indicated that the extreme case of hyper parameter p ($p = 1$ or $p = \infty$) might not be the optimal setting, we would like to introduce *standard gradient regularization* where $p = 2$. Then, we will develop a second order Taylor expansion analysis of this perturbation to provide further theoretical insight into this particular case. First, let us write down the formula for ϵ :

$$\begin{aligned}\epsilon &= \sigma \operatorname{sign}(\nabla \mathcal{L}) \left(\frac{|\nabla \mathcal{L}|}{\|\nabla \mathcal{L}\|_2} \right)^1 \\ &= \sigma \frac{\nabla \mathcal{L}}{\|\nabla \mathcal{L}\|_2}\end{aligned}\quad (12)$$

We are now ready to compute the second order expansion. The second order Taylor expansion of loss function is:

$$\mathcal{L}(x + \epsilon) \approx \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{1}{2} \epsilon^T H_{\mathcal{L}}(x) \epsilon \quad (13)$$

$$\approx \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{1}{2} \epsilon^T J_y(x) H_{\mathcal{L}}(y) J_y^T(x) \epsilon \quad (14)$$

(by Lemma B.1)

$$= \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{1}{2} \epsilon^T \nabla_x \mathcal{L} \nabla_x \mathcal{L}^T \epsilon \quad (15)$$

$$= \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{\sigma^2}{2} \frac{\nabla \mathcal{L}^T \nabla \mathcal{L} \nabla \mathcal{L}^T \nabla \mathcal{L}}{\|\nabla \mathcal{L}\|_2^2} \quad (16)$$

$$= \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{\sigma^2}{2} \nabla \mathcal{L}^T \nabla \mathcal{L} \quad (17)$$

$$= \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{\sigma^2}{2} \operatorname{Tr}(\nabla \mathcal{L} \nabla \mathcal{L}^T) \quad (18)$$

$$= \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{\sigma^2}{2} \operatorname{Tr}(J_y(x) H_{\mathcal{L}}(y) J_y^T(x)) \quad (19)$$

(by Lemma B.1)

$$\approx \mathcal{L} + \sigma \|\nabla_x \mathcal{L}\|_2 + \frac{\sigma^2}{2} \operatorname{Tr}(H_{\mathcal{L}}(x)) \quad (20)$$

Therefore, the second order Taylor expansion resembled the term induced by marginalizing over Gaussian noise.

Remark 2. It is possible to write down a generalized form of second order regularization term as $\frac{\sigma^2}{2} \|\nabla_x \mathcal{L}\|_{p^*}^2$. In this form, it does not provide further insight, as it appears to be duplication of the first order term in general.

E. Exact Solution Is Non-Trivial

As mentioned earlier, the original minmax problem (1) is generally difficult to solve. However, it would be still of interest to see whether the original minmax problem could be solved without using approximation. It turns out that it is not easy to achieve, even in the simplest case of a linear regression mode. We show this in the following.

The optimization problem of finding the worst perturbation in the linear regression model can be formulated as follows:

$$\max_{\epsilon} \|\mathbf{t} - \Theta(\mathbf{x} + \epsilon)\| \quad \text{s.t.} \quad \|\epsilon\|_p \leq \sigma \quad (21)$$

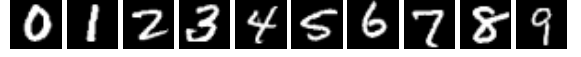


Fig. 1. Original input images from MNIST dataset



Fig. 2. Inputs perturbed by gradient perturbation with $p = 2$ and $\sigma = 1$

where $\mathbf{t}$ is the desired output, and Θ is the weight matrix. Hence, we are maximizing a convex function, which is non-trivial. One might hope that by applying the first order expansion, it could lead to a simple solution. However, while this approximately solves the maximization problem, the minimization problem becomes non-convex due to the additional regularization term. Therefore, it appears to be hard to obtain exact solution in this very simple model.

F. Computational Cost

The computational cost of injecting such adversarial noise comes from computing ϵ , which requires computing $\nabla_x \mathcal{L}$ and a minimum overhead to convert $\nabla \mathcal{L}$ into ϵ . A naive BP approach to compute ϵ will roughly double the time cost of the training. We implemented the algorithm using built in functionality provided by [11].

III. VISUALIZATION AND INTERPRETATION OF GRADIENT PERTURBATION

Traditionally, adversarial examples and corresponding perturbations are regarded as unintelligible to naked eyes [1], [2]. However, we will visualize our gradient perturbations in the case of $p = 2$, and thus provide both physical intuition of gradient perturbation and mathematical structure behind. The physical intuition, in particular, could support the effectiveness of gradient perturbation, and explain why adversarial examples could generalize across models. The model used to generate sample images in this section is a sigmoid network model trained on the MNIST dataset [8].

A. Visualizing Adversarial examples

It has been demonstrated that adversarial examples are generated by imperceptible or unrecognizable perturbations [1], [2]. However, we will show that in some cases we could visualize those perturbations. Let us present randomly picked samples from MNIST in Figure 1 and the perturbed images by gradient perturbation with $p = 2$ and $\sigma = 1$ in Figure 2.

Indeed, at the first glance, we may conclude that the perturbed examples are indistinguishable from the original copies. However, let us take a closer look at the perturbations in Figure 3 and Figure 4 by magnifying the perturbation with a factor of 10: Although the magnified perturbations do not still make much sense to naked human eye, the exaggerated adversarial examples are presented in a more meaningful way for the naked eye (see Figure 4). The perturbations have changed the physical shapes of the objects in a perceptible way. The number 2, 3, 6 are of special interest. The bottom right corner of 2 has been erased and turned into 7; 3 is extended into number 8; the 6 is the most interesting, where a part is erased while another part is extended to become a 5.



Fig. 3. The corresponding perturbations magnified by a factor of 10



Fig. 4. Inputs perturbed by gradient perturbation with $p = 2$ and $\sigma = 10$

B. Mathematical Structure

In this subsection, we will try to interpret the above phenomenon mathematically. To understand how those adversarial examples are generated, let us decompose the gradient perturbation ϵ when $p = 2$:

$$\epsilon = \sigma \frac{\nabla_x \mathcal{L}}{\|\nabla_x \mathcal{L}\|_2} \quad (22)$$

$$= \frac{\sigma}{\|\nabla_x \mathcal{L}\|_2} \nabla_x \mathcal{L} \quad (23)$$

$$= \frac{\sigma}{\|\nabla_x \mathcal{L}\|_2} \nabla_y \mathcal{L} J_y(x) \quad (24)$$

$$= \frac{\sigma}{\|\nabla_x \mathcal{L}\|_2} (\mathbf{y} - \mathbf{t}) J_o(x) \quad (25)$$

and where $\mathbf{o}$ is the input of the softmax layer for a typical neural network, and $\mathbf{t}$ is the true label. Next, we need to make the following assumption: the Jacobian matrix encodes the local version of corresponding classes. Then, when the prediction is concentrated on the correct output, $(\mathbf{y} - \mathbf{t})$ should be small and negative. This corresponds to the cases like 0, 1, and 9 in the previous section, where the perturbed examples are still clear and correct. In the case of low predication confidence, fuzzy perturbed examples could be generated. This corresponds to the cases like 4, 5, 7, and 8. The most interesting cases happen when there is a confusion between two classes. That is to say, $(\mathbf{y} - \mathbf{t})$ contains two large components where one component is positive and one negative. The negative component corresponds to weak prediction of correct label, and the positive component corresponds to most confused wrong prediction. Images like 2, 3, and 6 in the previous section are somewhat confusing when compared to the other numbers. In such cases, an adversarial perturbation erases the correct objects and injects the confused one.

Remark 3. In the above argument, we made the assumption of the Jacobian's ability to model the local space. This assumption is reasonable, since neurons in several layers of convolutional neural network correspond to certain natural image statistics [12]. While such assumption might not hold strictly in more complex data or data in other domains, there are researches indicate that the encoding of such information exist [13].

The above finding suggests that gradient perturbation method works in a distinctive way from techniques like weight decay or weight constraint method. It appears that gradient perturbation adds regularization effects conditioned on the performance of model in a particular instance. Therefore, it will not over-regularize when unnecessary. This interpretation is consistent with arguments made in [2].

C. Generalization of Adversarial Examples

The visualization tells us that image instances are actually morphed into other classes in a very minimal form. Our machine learning models are much more perceptive than people in detecting such minor perturbations. Thus, the ability of adversarial examples to generalize across different models might come from meaningful changes in examples. A small change towards a specific class will still be a change even added in another instance.

This explanation is also in coherent with [2]. In their framework, we can say that the generalization of adversarial examples is due to the models' resemblance of linear classifier, and linear classifier could encode local semantic information of images.

IV. EXPERIMENTS

In this section, we will apply our proposed gradient perturbation method on standard benchmarks including MNIST [8] and CIFAR-10 [14]. In addition, we will test whether gradient regularization could improve the models' robustness. We will mainly test the case for $p = 2$, which we refer to as *standard gradient regularization*.

A. MNIST

The MNIST dataset [8] consists of 60,000 training examples and 10,000 testing examples. Each single example consists of 28×28 greyscale images, which corresponds to digits from 0 to 9. We rescale the inputs into the range of $[0, 1]^{784}$, and no further preprocessing is applied. We tested *standard perturbation regularization* on three architectures: standard sigmoid multilayer perceptron (MLP), Maxout network, and Convolutional Maxout network [15]. The sigmoid MLP experiments are conducted to investigate the utility of our method on highly non-linear models, and the other two experiments are designed to further verify whether our proposed framework could obtain the state-of-the-art performance.

The sigmoid MLP has two hidden layers, and its parameters are chosen from [400,400], [600,600] and [800,800] hidden units based on validation. Only norm constraint is used as other means for regularization. The max norm is set to be the square root of 15 as in [16]. The regularization parameter σ is chosen from 0.1, 0.5, 1, and 2. The final architecture used is two layers of 600 sigmoid units, and a softmax final layer. σ is set to be 1. To fully utilize the training set, we followed [15] to train the model on 50,000 examples first, and we record the loss in the training set. Next, we train with a total of 60,000 examples until the last 10,000 examples hit the same loss.

For the experiments based on Maxout networks, we have engaged the model from Maxout paper [15] and applied gradient regularization where $\sigma = 1$ as we have found from experiments on sigmoid MLP. No other parameter setting is tested. In addition, we tested the case of $p = \infty$ for the Convolutional Maxout network, since it is not reported by [2]. We have found that setting $\sigma = 0.1$ leads to better results than $\sigma = 0.25$, which is the best parameter for the non-convolutional Maxout network according to [2].

²It is recently improved to 0.78% by applying adversarial idea on early stopping, and changing architecture.

TABLE I. TESTING ERRORS ON PERMUTATION INVARIANT MNIST DATASET WITHOUT DATA AUGMENTATION.

Models	Test Error
Maxout + dropout [15]	0.94 %
Sigmoid mlp + gradient $p = 2$	0.93 %
ReLU + dropout + gaussian [17]	0.85 %
Maxout + dropout + gradient $p = \infty$ [2]	0.84 % ²
DBM + dropout [18]	0.79 %
Maxout + dropout + gradient $p = 2$	0.78 %

TABLE II. TESTING ERRORS ON MNIST DATASET WITHOUT DATA AUGMENTATION. CONVOLUTIONAL ARCHITECTURE IS USED.

Models	Test Error
Conv. Maxout + dropout [15]	0.45 %
Conv. Maxout + dropout + gradient $p = \infty$	0.41 %
Conv. Maxout + dropout + gradient $p = 2$	0.39 %
Conv. Kernel Network [19]	0.39 %

In the permutation invariant version, we achieved 78 errors among 10, 000 test samples in the case of $p = 2$. **To our best knowledge, this is the best result in this category even with unsupervised pretraining if data are not augmented.** Also, gradient regularized old fashioned sigmoid MLP has been shown to achieve 93 errors, which actually beats simple Maxout networks with dropout training. This indicates that even a highly nonlinear model such as sigmoid MLP could be linear enough locally to benefit from gradient regularization. We summarized our results and other related best results in Table I.

In terms of the improvements on convolutional architecture, our best result is obtained by standard gradient regularization. We have achieved 39 errors on testing set, which ties with the recently proposed sophisticated convolutional kernel network described by [19]. To our best knowledge, no better results are obtained without using of data augmentation like elastic distortion. It appears that standard gradient regularization outperforms fast gradient sign method in a consistent fashion, especially if we consider that the σ parameter is not tuned for Maxout network. We summarized the results in Table II.

B. CIFAR-10

The CIFAR-10 dataset [14] consists of 60,000 32×32 RGB images, corresponding to 10 categories of objects. There are 50,000 training examples and 10,000 testing examples. Again, we have followed the procedure in the Maxout paper [15]. To quickly verify gradient regularization in a more complex setting, we tested on a small convolutional maxout network reported by [20], and can confirm again the improvement of our regularization technique. σ was set to be $1 \times \sqrt{\frac{32^2 \times 3}{28^2}} \approx 2$ in this case, since the RBG image has more dimensions. Similar to MNIST, the training has two stages,

TABLE III. TESTING ERRORS ON CIFAR-10 DATASET WITHOUT DATA AUGMENTATION.

Models	Testing Error
Conv. Maxout Small + dropout [20] (40K)	14.05 %
Conv. Maxout Small + dropout + Gradient $p = 2$ (40K)	13.26 %
Conv. Maxout Small + dropout [20] (50K)	12.93 %
Conv. Maxout Small + dropout + Gradient $p = 2$ (50K)	12.28 %

TABLE IV. TESTING ERRORS OF MNIST DATASET UNDER GAUSSIAN NOISES

Models	Gaussian std		
	0	0.1	0.3
Conv. Maxout + dropout	0.45 %	0.76 %	20.17 %
Maxout + dropout	0.94 %	1.14 %	2.42 %
Conv. Maxout + dropout + Gradient $p = 2$	0.39 %	0.44 %	14.36 %
Maxout + dropout + Gradient $p = 2$	0.78 %	0.83 %	1.29 %

we compared the test error in the end of both stages. As can be seen from Table III, our gradient regularization technique improves the performance of this network.

C. Robustness

A recent theoretical study suggests there is a relationship between adversarial examples and Gaussian noise [6], we have used testing errors under Gaussian noise as a criterion to test whether our regularization technique improves the models' robustness against perturbations. We will also discuss the relationship in V. We gather some data based on testing errors on MNIST.

As can be seen from Table IV and V, our regularization technique indeed improves the models' ability to resist perturbation. However, it is interesting to note that the convolutional architecture seems to suffer more severe than the standard Maxout network. This observation is consistent with the data gathered from the ImageNet data set [1], where researchers have shown that the operator norm is larger in the convolutional architecture than fully connected networks. This is however yet another indication that the randomness in natural images is quite small even compare to those so called imperceptible perturbations.

TABLE V. TESTING ERRORS OF CIFAR DATASET UNDER GAUSSIAN NOISES

Models	Gaussian std		
	0	0.1	0.3
Conv. Maxout + dropout	13.08 %	13.37 %	16.88 %
Conv. Maxout + dropout + Gradient $p = 2$	12.28 %	12.81 %	16.80 %

V. DISCUSSION

In this section, we will show how to exploit the proposed unified theory and mainly discuss why adversarial examples do not occur frequently in real data. Some of the following analysis may not be strict enough but sufficient to provide intuition and interpretations on certain properties of adversarial examples.

By assuming the perturbation is caused by the Gaussian noise, we will first analyze how the misclassification rates of various learning models could be related to the minimum perturbation. Here, the minimum perturbation is measured in terms of its perturbation size that is able to change the models' prediction on a certain example. After the establishment of the relationship, we will be able to predict the probability or the frequency that adversarial examples occur in real data. In order to achieve these objectives, we will develop a mathematical model by following two fundamental assumptions made in [2]: the manifold where adversarial examples live is not special and they mainly arise from local linear behavior.

More precisely, we assume that every data point is associated with a Gaussian distribution: $(\mathbf{x} + \boldsymbol{\eta}) \sim \mathcal{N}(\mathbf{x}, \sigma^2 \mathbf{I}_d)$ where σ is the standard deviation in every dimension, and $\mathbf{I}_d$ is an identity matrix of size $d \times d$. Thus, adversarial examples do not live in a region with particularly low probability as contrary to suggestion from [1]. There are n directions to generate effective adversarial examples, whose corresponding minimum perturbations are respectively denoted as $\mathbf{a}_i$, and direction $\mathbf{e}_i = \frac{\mathbf{a}_i}{\|\mathbf{a}_i\|}$ for $i = 1, 2, \dots, n$. In order to generate an adversarial example from $\boldsymbol{\eta}$, we need to satisfy:

$$\exists i, \quad \text{s.t.} \quad \boldsymbol{\eta}^T \mathbf{e}_i \geq \mathbf{a}_i^T \mathbf{e}_i \quad (26)$$

Denote adversarial examples to occur as A , then we could obtain a lower bound:

$$P(A) \leq \sum_{i=1}^n P(\boldsymbol{\eta}^T \mathbf{e}_i \geq \mathbf{a}_i^T \mathbf{e}_i) \quad (27)$$

Since the Gaussian distribution is isotropic, $P(\boldsymbol{\eta}^T \mathbf{e}_i \geq \mathbf{a}_i^T \mathbf{e}_i)$ could be simplified into:

$$P(\boldsymbol{\eta}_0 \geq \|\mathbf{a}_i\|) \quad (28)$$

and where $\boldsymbol{\eta}_0$ means $\boldsymbol{\eta}$ in an arbitrary axis. This essentially means that given $\|\mathbf{a}_i\|$, it will not increase the probability of adversarial examples to occur by increasing dimension of input space. To calculate the probability, we only need $\|\mathbf{a}_i\|$. To further simplify our formula, we assume all $\|\mathbf{a}_i\|$'s are equal, and just denote each $\mathbf{a}_i$ as $\mathbf{a}$. Hence, there are three parameters in our model: $n, \|\mathbf{a}\|, \sigma$, among which the size of σ is controlled.

We could develop some qualitative intuitions first. To make a fair comparison, we set that the mean distortion of adversarial examples, as defined by [1], equal to the variance of the Gaussian distribution. In other words, we set $\sigma^2 = \frac{\|\mathbf{a}\|_2^2}{d}$, so that our model only depends on the number of directions n and the dimension of input d . Then, based on Equation (28), we have:

$$P(A) \leq nP\left(\frac{\boldsymbol{\eta}_0}{\sigma} \geq \sqrt{d}\right) \quad (29)$$

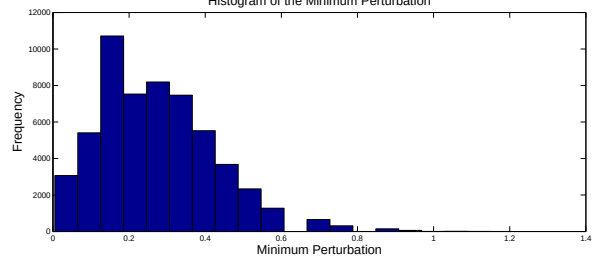


Fig. 5. Histogram of minimum perturbation in a single layer softmax model with regularization parameter $\lambda = 1e-4$. Histograms of other regularization parameters are of the same shape but different in scale

Since the event denotes $\boldsymbol{\eta}_0$ to be $\sqrt{d}$ standard deviation away from the mean, the probability of the event to occur shrinks exponentially. Therefore, while high dimensionality makes it possible to find a direction such that the model prediction would be changed, it also provides protection in a probabilistic sense that such prediction change could be unlikely.

In the next subsection, we will concretely take the MNIST data as one real example to make further analysis. The image pixels used in the following analysis are all scaled into $[0,1]$. Following [1], we will conduct our analysis based on the simple softmax model, which is of linear nature.

A. Estimating Misclassification Rate under Gaussian Noise

This subsection mainly aims at verifying our linear analyzing framework and gathering insights from empirical observations. To get a reasonable numerical estimation of misclassification rate, it is not enough to use the average of minimum perturbation $\|\mathbf{a}\|_2$, instead we have to take the distribution of minimum perturbation into consideration. There are two reasons for this: (1) the minimum perturbations of a small value have a major impact on the probability that adversarial examples occur, and (2) the minimum perturbation skews towards small values as seen in Figure 5. We approximate the distribution of $\|\mathbf{a}\|$ by a Gaussian distribution $\mathcal{N}_a(\mu_a, \sigma_a^2)$. We will then have:

$$P(A) \leq nP(\boldsymbol{\eta}_0 \geq \|\mathbf{a}\|) \quad (30)$$

$$= nP(\boldsymbol{\eta}_0 - \|\mathbf{a}\| \geq 0) \quad (31)$$

The summation of two Gaussian random variables, denoted by $\delta \equiv \boldsymbol{\eta}_0 - \|\mathbf{a}\|$, is Gaussian, whose mean and variance are $-\mu_a$ and $\sigma^2 + \sigma_a^2$ respectively. Then, we have the probability related to a cumulative distribution function of a Gaussian variable:

$$P(A) \leq nP(\delta \geq 0) \quad (32)$$

If we further assume the smallest perturbation in one certain direction dominates the predicated probability, we can roughly set $n = 1$.³ Then, the remaining task is to get an estimation of μ_a and σ_a^2 . Following the linear view of adversarial examples, we perform a line-search in the direction of gradient $\nabla_{\mathbf{x}} \mathcal{L}$ to find the minimum perturbation needed to change the prediction from correct to wrong. We have gathered the statistics from

³Such assumption may be less strict, which is however simple and useful to get clear insight into the theory.

TABLE VI. MEAN AND STANDARD DEVIATION OF THE MINIMUM PERTURBATIONS FOR SOFTMAX MODELS ON MNIST TRAINING DATASET

Models	mean	standard deviation
Softmax $\lambda = 1e - 4$	0.2744	0.1511
Softmax $\lambda = 1e - 2$	0.4722	0.2531
Softmax $\lambda = 1$	1.2718	0.6654

TABLE VII. ACUTAL MISCLASSIFICATION RATE ON MNIST UNDER THE PERTURBATION OF GAUSSIAN NOISE

Models	Gaussian std 0	0.1	0.3
Softmax $\lambda = 1e - 4$	6.02 %	11.99 %	40.07 %
Softmax $\lambda = 1e - 4$	6.29 %	8.22 %	22.44 %
Softmax $\lambda = 1e - 4$	8.93 %	9.16 %	11.33 %

TABLE VIII. MISCLASSIFICATION RATE THEORETICALLY ESTIMATED BY (33) ON MNIST UNDER THE PERTURBATION OF GAUSSIAN NOISE

Models	Gaussian std 0	0.1	0.3
Softmax $\lambda = 1e - 4$	6.02 %	12.12 %	25.47 %
Softmax $\lambda = 1e - 2$	6.29 %	10.16 %	17.01 %
Softmax $\lambda = 1$	8.93 %	11.61 %	12.63 %

softmax models, and summarized the results in Table VI. Note that these statistics are obtained under the training set (with sufficiently large number of samples) so as to get a statistically reliable conclusion. Similar inspection can be derived if the test data were used.

As can be seen from Table VI, a larger regularization parameter corresponds to better robustness. To estimate the misclassification rate under different levels of Gaussian noises, we first need to obtain the misclassification rate without noise. With the further assumption that misclassification is not corrected by noise, the predicated misclassification rate can be calculated theoretically as:

$$P(\text{miss}) + (1 - P(\text{miss})) \times P(\delta \geq 0) \quad (33)$$

Given the perturbation under the Gaussian noise, we can get the actual misclassification rates in MNIST and also estimate such rate theoretically using Equation (33). These results are listed in Table VII and Table VIII respectively.

As can be seen by comparing Table VII and Table VIII, the misclassification rate estimated from our probability model fits well the actual rate as obtained from empirical experiments. Although these numerical results do not match exactly as seen in Table VII and Table VIII, the qualitative changes of the predications and actual results are fairly consistent under different levels of Gaussian noises.

Note that, within the linear view of adversarial examples, we have made two major assumptions in the previous analysis. First, we adopt the most dominant direction to estimate the misclassification rate. This may lead to underestimation of misclassification, since other directions are neglected. Second, we approximate the distribution of minimum perturbation by a Gaussian distribution. This may lead to overestimation of misclassification rate in the case of no noise. Table VI indicates that standard deviation of minimum is roughly half of the mean



Fig. 6. Images corrupted by Gaussian noise with $\sigma = 0.01$

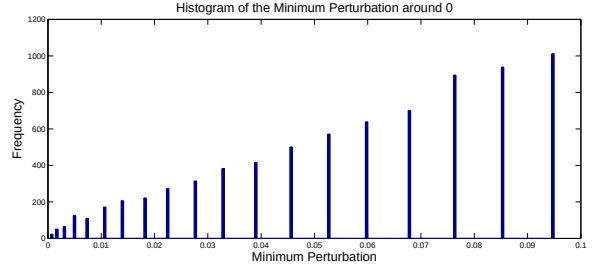


Fig. 7. Histogram of minimum perturbation in a single layer softmax model with regularization parameter $\lambda = 1e - 4$. Region beyond 0.1 is filtered.

in a consistent fashion. Then, even without Gaussian noise, the calculation yields roughly $P(A) \approx 3\%$. The reason behind this is that the actual distribution is only supported in the positive region, but Gaussian distribution is not. This simplification does not have a major impact when σ is large, and our results agree with this. However, to estimate $P(A)$ when σ is small, we may need to readjust our approximation.

B. Estimating the Probability of Adversarial Examples

To estimate the probability that adversarial examples occur in reality, we need to estimate the randomness in natural images and the distribution of the minimum perturbations. This is especially necessary when the perturbation is sufficiently small, i.e., close to 0, since the overestimation occurs in this case as indicated in the previous subsection. In the following, we intend to choose a σ that is sufficiently large, yet is hardly perceptible. We first illustrate some examples from MNIST dataset in Figure 6, where $\sigma = 0.01$.

In Figure 6, such noise is imperceptible even with a close examination, and this is not the case for significantly larger σ . We observe that the distribution of the minimum perturbation appears to be linear when it is close to zero as seen in Figure 7.⁴ In light of this, we could assume the probability density is linear around 0 with respect to the minimum perturbation. After gathering statistics from the graph and some algebras, we can predicate the increase of misclassification rate in theory (based on our model) for a perturbation caused by Gaussian noise with $\sigma = 0.01$. The percentage of such additional misclassification is calculated theoretically and listed in Table IX. We also list the actual additional misclassification, which was obtained empirically from experiments. Note that, according to the above analysis and assumptions, these additional misclassified samples can be regarded as adversarial examples that occur naturally, when a perturbation is given by Gaussian noise. As seen in Table IX, these numbers are of sufficiently small both theoretically (under column ‘‘Predicated’’) and empirically (under column ‘‘Actual’’), meaning that the probability of adversarial examples is quite small. Therefore, the claim that adversarial examples

⁴The space in the figure is caused by steps in line search.

TABLE IX. ACTUAL AND PREDICATED PERCENTAGE OF ADDITIONAL MISCLASSIFICATION RATE CAUSED BY THE PERTURBATION OF GAUSSIAN NOISE WITH $\sigma = 0.01$ ON MNIST

Models	Actual	Predicated
Softmax $\lambda = 1e - 4$	0.07 %	0.04 %
Softmax $\lambda = 1e - 2$	0.01 %	0.02 %
Softmax $\lambda = 1$	0.01 %	0.01 %

could endanger our machine learning models seems to somewhat over cautious, since its natural occurrence is supposed to be rare and negligible comparing to classification errors of our models. This is both verified theoretically and empirically in the above analysis.

VI. CONCLUSION

In this paper, we have proposed a unified framework, which aims at building robust models against adversarial examples. We have derived a family of gradient regularization techniques, which could be extensively used in various machine learning models. In particular, a special case has a second order interpretation, which approximately marginalized over Gaussian noise. We explained the adversarial examples' ability to generalize across models by visualizing its effects, which had been previously thought to be unrecognizable. This visualization, in turn, supplement our regularization technique with physical meaning.

While the linear view appears to be plausible and fruitful, so far, it needs more empirical support. To be more concrete, manifold hypothesis suggests data residue in a low dimensional manifold in the space. This, however, suggests that it is always possible to leave the manifold by slight change, since there are extra dimension around every point in the manifold, and the fact supports the hypothesis. Theoretically, a low dimensional manifold has measure of zero. This makes the number analogy made by [2] almost literally true, but with adversarial examples being irrational numbers. In reality, however, data float around the manifold. Thus, geometrically, we could check the implication of linear view by associating errors induced by Gaussian noises and the minimum perturbation length. Our primary work suggests that it would require distribution of minimum perturbation length to account for the actual errors.

ACKNOWLEDGMENT

The authors would like to thank the developer of Deeplearning Toolbox [21], the developers of Theano [11], [22] and the developers of Pylearn2 [20].

APPENDIX A SOLVING ϵ

Since $\mathcal{L}(x)$ is independent of ϵ . Our problem is

$$\max_{\epsilon} \nabla_x \mathcal{L}^T \epsilon \quad \text{s.t.} \quad \|\epsilon\|_p \leq \sigma \quad (34)$$

Clearly, the optimal ϵ would have a norm of σ , otherwise, we can normalize ϵ to get a greater loss. Therefore, we are set to solve

$$\max_{\epsilon} \nabla_x \mathcal{L}^T \epsilon \quad \text{s.t.} \quad \|\epsilon\|_p = \sigma \quad (35)$$

This could be solved by standard Lagrangian multiplier method, where $f(\epsilon) \equiv \nabla_x \mathcal{L}^T \epsilon$, and $g(\epsilon) \equiv \|\epsilon\|_p = \sigma$. Set $\nabla f(\epsilon) = \lambda \nabla g(\epsilon)$, we have

$$\nabla f(\epsilon) = \lambda \nabla g(\epsilon) \quad (36)$$

$$\nabla_x \mathcal{L} = \lambda \frac{\epsilon^{p-1}}{p(\sum_i \epsilon_i^p)^{1-\frac{1}{p}}} \quad (37)$$

$$\nabla_x \mathcal{L} = \frac{\lambda}{p} \left(\frac{\epsilon}{\sigma}\right)^{p-1} \quad (38)$$

$$\nabla_x \mathcal{L}^{\frac{p}{p-1}} = \left(\frac{\lambda}{p}\right)^{\frac{p}{p-1}} \left(\frac{\epsilon}{\sigma}\right)^p \quad (39)$$

$$\quad (40)$$

Sum over two sides

$$\sum \nabla_x \mathcal{L}^{\frac{p}{p-1}} = \sum \left(\frac{\lambda}{p}\right)^{\frac{p}{p-1}} \left(\frac{\epsilon}{\sigma}\right)^p \quad (41)$$

$$\|\nabla \mathcal{L}\|_{p^*}^{p^*} = \left(\frac{\lambda}{p}\right)^{p^*} * 1 \quad (42)$$

$$\left(\frac{\lambda}{p}\right) = \|\nabla \mathcal{L}\|_{p^*} \quad (43)$$

Combine (38) and (43), it is easy to see

$$\epsilon = \sigma \text{sign}(\nabla \mathcal{L}) \left(\frac{\|\nabla \mathcal{L}\|_{p^*}}{\left(\frac{\lambda}{p}\right)^{\frac{1}{p-1}}}\right)^{\frac{1}{p-1}} \quad (44)$$

APPENDIX B LEMMA

Let vector y be the output of model, i.e. $y = y(x)$. Correspondingly, we have vector t as the desired output. We have N as the length of y , where N is the number of classes in a classification task. $GN(x) \equiv J_y(x)H_{\mathcal{L}}(y)J_y^T(x)$ is Gaussian-Newton matrix, which is an approximation of Hessian matrix of $\mathcal{L}$ w.r.t. x [23]. The $J_y(x)$ is the Jacobian matrix of y w.r.t. x , and $H_{\mathcal{L}}(y)$ is the Hessian matrix of $\mathcal{L}$ w.r.t. y .

Lemma B.1. Given $\mathcal{L} = -\sum_{i=1}^N t_i \log(y_i)$, and $\exists l \in 1, 2, \dots, N$ s.t. $\forall i \in 1, 2, \dots, N, i \neq l, i = \delta_{li}$. We have $\nabla_x \mathcal{L} \nabla_x \mathcal{L}^T = J_y(x)H_{\mathcal{L}}(y)J_y^T(x)$.

Proof: First, by chain rule, we have $\nabla_x \mathcal{L} = J_y(x) \nabla_y \mathcal{L}$. By cancellation of $J_y(x)$ on both sides, it suffices to show $\nabla_y \mathcal{L} \nabla_y \mathcal{L}^T = H_{\mathcal{L}}(y)$. Now, let us calculate the gradient:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial y_i} &= -\frac{t_i}{y_i} \\ &= -\frac{\delta_{il}}{y_i} \end{aligned} \quad (45)$$

Then, compute the out product: $(\nabla_y \mathcal{L} \nabla_y \mathcal{L}^T)_{ij} = \frac{\delta_{il} \delta_{jl}}{y_i y_j}$. This means all the off-diagonal items are zeros, and only one term left in the diagonal, which is $(\nabla_y \mathcal{L} \nabla_y \mathcal{L}^T)_{ll} = \frac{1}{y_l^2}$. Let us compute Hessian matrix:

$$\begin{aligned} \frac{\partial^2 \mathcal{L}}{\partial y_j \partial y_i} &= -\frac{\partial}{\partial y_j} \frac{\delta_{il}}{y_i} \\ &= \frac{\delta_{il} \delta_{ij}}{y_i^2} \end{aligned} \quad (46)$$

Hence, $H_{\mathcal{L}}(y)$ also have only one diagonal term, which is $(H_{\mathcal{L}}(y))_{ii} = \frac{1}{y_i^2}$. Therefore, we have the equality as desired. ■

REFERENCES

- [1] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” *CoRR*, vol. abs/1312.6199, 2013. [Online]. Available: <http://arxiv.org/abs/1312.6199>
- [2] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” *arXiv preprint arXiv:1412.6572*, 2014.
- [3] A. Nguyen, J. Yosinski, and J. Clune, “Deep neural networks are easily fooled: High confidence predictions for unrecognizable images,” *arXiv preprint arXiv:1412.1897*, 2014.
- [4] F. Wang, W. Liu, and S. Chawla, “On sparse feature attacks in adversarial learning,” in *Data Mining (ICDM), 2014 IEEE International Conference on*, Dec 2014, pp. 1013–1018.
- [5] D. Lowd and C. Meek, “Adversarial learning,” in *Proceedings of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery in Data Mining*, ser. KDD ’05. New York, NY, USA: ACM, 2005, pp. 641–647. [Online]. Available: <http://doi.acm.org/10.1145/1081870.1081950>
- [6] A. Fawzi, O. Fawzi, and P. Frossard, “Analysis of classifiers’ robustness to adversarial perturbations,” *CoRR*, vol. abs/1502.02590, 2015. [Online]. Available: <http://arxiv.org/abs/1502.02590>
- [7] S. Gu and L. Rigazio, “Towards deep neural network architectures robust to adversarial examples,” *arXiv preprint arXiv:1412.5068*, 2014.
- [8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [9] C. M. Bishop, “Training with noise is equivalent to tikhonov regularization,” *Neural Computation*, vol. 7, pp. 108–116, 1994.
- [10] P. Sermanet, S. Chintala, and Y. LeCun, “Convolutional neural networks applied to house numbers digit classification,” in *Pattern Recognition (ICPR), 2012 21st International Conference on*. IEEE, 2012, pp. 3288–3291.
- [11] J. Bergstra, O. Breuleux, F. Bastien, P. Lamblin, R. Pascanu, G. Desjardins, J. Turian, D. Warde-Farley, and Y. Bengio, “Theano: a CPU and GPU math expression compiler,” in *Proceedings of the Python for Scientific Computing Conference (SciPy)*, Jun. 2010, oral Presentation.
- [12] M. D. Zeiler and R. Fergus, “Visualizing and understanding convolutional networks,” in *Computer Vision–ECCV 2014*. Springer, 2014, pp. 818–833.
- [13] A. Dosovitskiy and T. Brox, “Inverting convolutional networks with convolutional networks,” *CoRR*, vol. abs/1506.02753, 2015. [Online]. Available: <http://arxiv.org/abs/1506.02753>
- [14] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” *Computer Science Department, University of Toronto, Tech. Rep.*, vol. 1, no. 4, p. 7, 2009.
- [15] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, “Maxout Networks,” Sep. 2013. [Online]. Available: <http://arxiv.org/abs/1302.4389>
- [16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, Jan. 2014. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2627435.2670313>
- [17] B. Poole, J. Sohl-Dickstein, and S. Ganguli, “Analyzing noise in autoencoders and deep networks,” *CoRR*, vol. abs/1406.1831, 2014. [Online]. Available: <http://arxiv.org/abs/1406.1831>
- [18] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Improving neural networks by preventing co-adaptation of feature detectors,” *CoRR*, vol. abs/1207.0580, 2012. [Online]. Available: <http://arxiv.org/abs/1207.0580>
- [19] J. Mairal, P. Koniusz, Z. Harchaoui, and C. Schmid, “Convolutional kernel networks,” in *Advances in Neural Information Processing Systems 27*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds. Curran Associates, Inc., 2014, pp. 2627–2635. [Online]. Available: <http://papers.nips.cc/paper/5348-convolutional-kernel-networks.pdf>
- [20] I. J. Goodfellow, D. Warde-Farley, P. Lamblin, V. Dumoulin, M. Mirza, R. Pascanu, J. Bergstra, F. Bastien, and Y. Bengio, “Pylearn2: a machine learning research library,” *arXiv preprint arXiv:1308.4214*, 2013. [Online]. Available: <http://arxiv.org/abs/1308.4214>
- [21] R. B. Palm, “Prediction as a candidate for learning deep hierarchical models of data,” 2012.
- [22] F. Bastien, P. Lamblin, R. Pascanu, J. Bergstra, I. J. Goodfellow, A. Bergeron, N. Bouchard, and Y. Bengio, “Theano: new features and speed improvements,” *Deep Learning and Unsupervised Feature Learning NIPS 2012 Workshop*, 2012.
- [23] N. N. Schraudolph, “Fast curvature matrix-vector products for second-order gradient descent,” *Neural Computation*, vol. 14, p. 2002, 2002.